

Translated

From: e-maxx.ru

Games on arbitrary graphs

Let a game be played by two players on an arbitrary graph G . I.e. the current state of the game is a certain vertex. The players perform moves by turns, and move from the current vertex to an adjacent vertex using a connecting edge. Depending on the game, the person that is unable to move will either lose or win the game.

We consider the most general case, the case of an arbitrary directed graph with cycles. It is our task to determine, given an initial state, who will win the game if both players play with optimal strategies or determine that the result of the game will be a draw.

We will solve this problem very efficiently. We will find the solution for all possible starting vertices of the graph in linear time with respect to the number of edges: $O(m)$.

Description of the algorithm

We will call a vertex a winning vertex, if the player starting at this state will win the game, if they play optimally (regardless of what turns the other player makes). Similarly, we will call a vertex a losing vertex, if the player starting at this vertex will lose the game, if the opponent plays optimally.

For some of the vertices of the graph, we already know in advance that they are winning or losing vertices: namely all vertices that have no outgoing edges.

Also we have the following **rules**:

- if a vertex has an outgoing edge that leads to a losing vertex, then the vertex itself is a winning vertex.
- if all outgoing edges of a certain vertex lead to winning vertices, then the vertex itself is a losing vertex.
- if at some point there are still undefined vertices, and neither will fit the first or the second rule, then each of these vertices, when used as a starting vertex, will lead to a draw if both player play optimally.

Thus, we can define an algorithm which runs in $O(n \cdot m)$ time immediately. We go through all vertices and try to apply the first or second rule, and repeat.